

CPP Project Parts 2 & 3

Chose Project: Cinematic - <https://github.com/dlvtlm/Cinematic>

We chose cinematic due to its simple and logical structure. With a good uml, we noticed it's straightforward while meeting nearly all project requirements. It even has two proper diamond inheritances. When we skimmed over the code, we noticed it had great potential, with a really good main, virtual functions specified, and much more.

Non-Chosen Projects:

1. Farmville - <https://github.com/einavh550/Farmville>

This project puzzled me, as I was very confused with the many decisions done during its writing. A bad uml was just the beginning, from missing move constructors and move assignments in the classes that need them, to all constructors being empty and all class variables are set through set functions. My greatest confusion was the fact that empty cpp files for each header were added.

2. SocialLens - <https://github.com/roylbrman-pixel/CPP-Project-Part1-SocialLens>

This project is not quite as bad as the last. While still having a bad uml and missing move constructors and move assignments, most other problems were minor. The one problem that really bugged me was the fact they used print inheritance, printSpecs is virtual in the parent and children implement it for unified print, but they both did not implement it correctly and didn't use it with operator<<.

Changes

Fix	Files [*.h]	Required?	Reason for fix	Possible reason for previous implementation
Inline Methods	Shift, Ticket, Hall, Hall3D, VIPHall, VIPTicket, Customer, Date, Cinema, Movie, Reviewer, Employee, Guest	No	Better readability and cleaner cpp files.	Original writers were not sure when inlining was acceptable and decided to not inline them at all.
PrintShift replaced with operator<<	Shift	No	operator<< is always better and cleaner.	Project requirements specify at least 5 operators, maybe they tried exactly 5.
PrintHall replaced with toOS and operator<<	Hall, VIPHall, Hall3D, Hall3DVIP,	No	operator<< is always better and cleaner, combined with logic for inherited print	This was not learned at the time
PrintDetails replaced with toOS and operator<<	Person, Guest, Employee, Customer, Reviewer	No	operator<< is always better and cleaner, combined with logic for inherited printcleaner.	This was not learned at the time
Return const in assignment operators	Cinema, Customer, Employee, Guest, Reviewer, Person, Movie	No	assignment operators should return const so there won't be multiple assignments happening simultaneously	They forgot
Remove move c'tor and d'tor	Shift, Ticket, VIPTicket, Hall, VIPHall, Hall3D, Hall3DVIP, Employee	No	no runtime memory allocation	They added them to all files to make sure they don't miss the files that actually need them
Set movies and shifts as const	Cinema	No	created shifts and movies in the cinema shouldn't be modified	They created all the arrays together at the start without realizing movies and shifts can't be changed
Update prints in main	main.cpp	No	changing print{Class} to toOs + operator<<	This was not learned at the time
Using namespace std in main	main.cpp	No	cleaner code, removes "std::"	They forgot

BirthDate property moved from Employee to Person, constructors in Person and its derived classes changed accordingly	Person, Employee, Customer, Guest, Reviewer, main	Yes	birthDate is a property that every person has, not just an employee	They didn't think it through
Set the return type of Prefix implementation to const	Employee	No	the Prefix implementation's return type should be const	They forgot
Changed constants from const to constexpr	Guest, Hall, Cinema, Ticket, main, Employee, VIPTicket	Yes	Because the value is initiated from the beginning it should be constexpr	They didn't think about it
Added override to child's overriding methods	Guest	No	to declare a method as overriding, it needs to have "override" keyword	They forgot
Give value to constexp static double	Employee, VIPTicket, Ticket	Yes	Valueless const static cant be used	They planned for the implementing team (part 2) to set the values, but forgot they are needed to be set in the header
Movie* move => const, and, currentMovie is const	Hall, Hall3D, VIPHall, Hall3DVIP, main	No	movies array in Cinema is declared as const	A chain reaction of our changes
Add Move c'tor and move operator=	Cinema, Movie	Yes	dynamic memory alloc = need copy, move, delete	They forgot
Delete copy c'tor and operator= (= delete)	Person	Yes	cant duplicate a human	An oversight on their part
Remove copy c'tor and operator= where uncesecary	Employee, Guest, Reviewer, Customer, Hall, Hall3D, Hall3DVIP	No	copy c'tor and op= were deleted from their ancestor	A chain reaction of our changes

Add Ticket (Movie) print to compareTicketPrices	main	No	can't compare tickets if we don't know the tickets	They forgot
---	------	----	--	-------------